



NILMBench2026

A deployment-aware benchmark for energy disaggregation

Aayush Kuloor* · Anurag Singh* · Harsh Dhru* · Nipun Batra

`{aayush.kuloor, anurag.s, harsh.dhru, nipun.batra}@iitgn.ac.in`

Indian Institute of Technology Gandhinagar

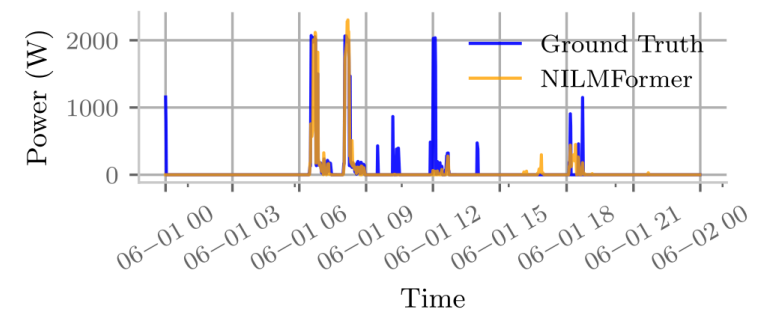
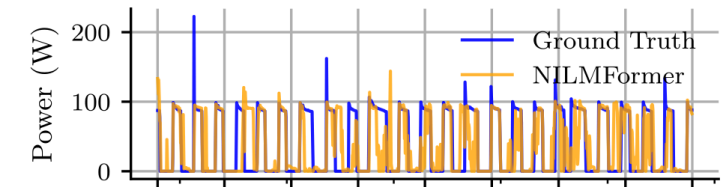
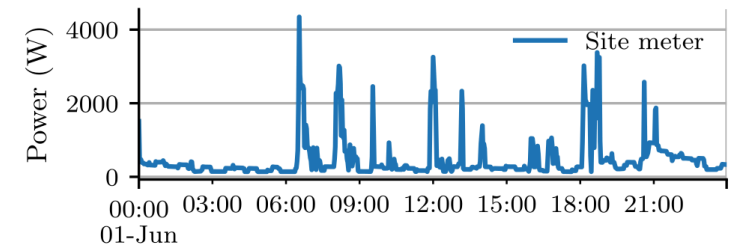
ACM BuildSys 2026 · Banff, Canada | **Best Paper Candidate** | *equal contribution

What is NILM, and why does it matter?

Non-Intrusive Load Monitoring (NILM) converts a **single smart-meter signal** into **appliance-level** consumption estimates.

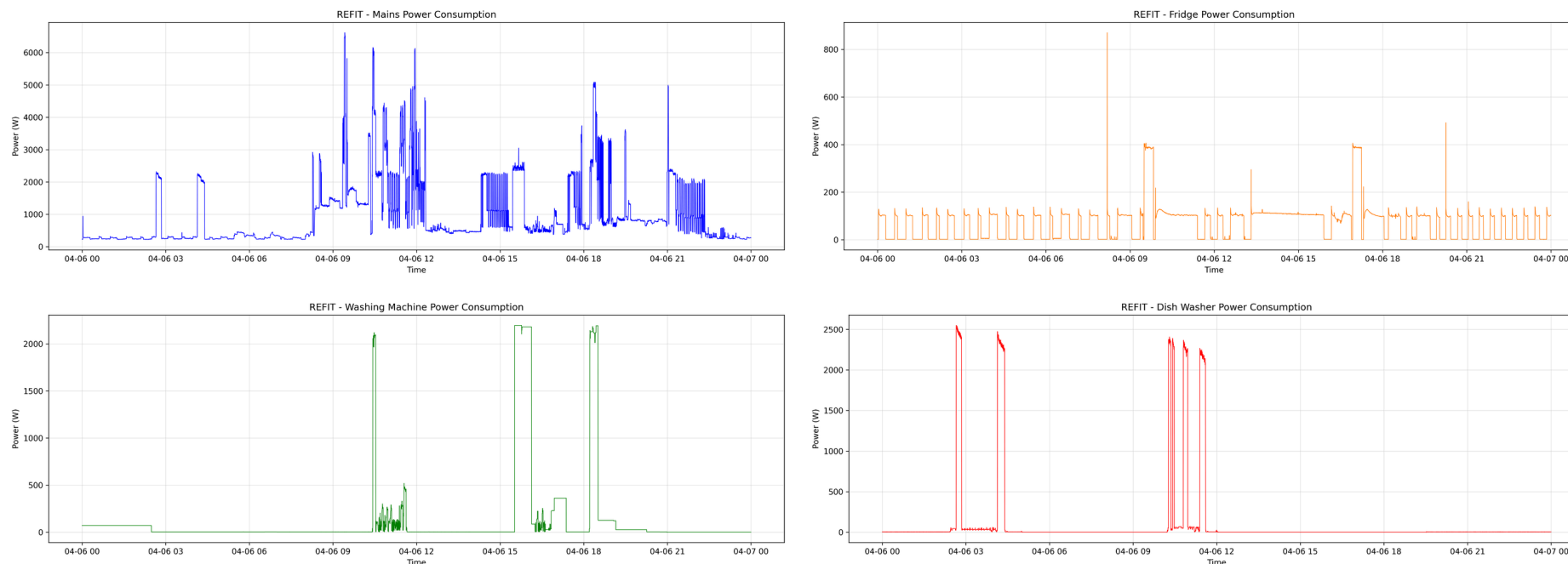
$$y_t = \sum_{i=1}^N x_{i,t} + \epsilon_t$$

- **Why it matters:** appliance-level feedback can reduce household consumption by up to **15%** — without installing a sensor on every device
- It is a hard **inverse problem**: appliance signatures vary across homes, making it a domain-adaptation challenge



UK-DALE: aggregate mains disaggregated into appliances

Appliance signatures make disaggregation possible



Each appliance has a distinct electrical fingerprint — periodic (fridge), multi-stage (washing machine), or sparse and high-power (dishwasher).

The evolution of NILM

1980s–90s

Combinatorial

Hart's edge detection & optimization

2000s

Probabilistic

Factorial Hidden Markov Models

2015 →

Deep learning

CNNs, RNNs (Kelly & Knottenbelt)

2020 →

Transformers

Long-range attention (NILMFormer)

As models grew more capable, evaluation did not keep pace. Benchmarking had to move **beyond accuracy** — to efficiency, resolution, and generalization.

What previous benchmarks missed

CAPABILITY	NILMTK 2014	CONTRIB 2019	NILMBENCH2026
Models	2	9	16
Temporal resolutions	variable	1 min	1 min and 15 min
Efficiency metrics	—	—	FLOPs, params, time
Cross-building test	—	yes	yes
Cross-dataset transfer	—	—	yes
Software stack	Python 2.7	TensorFlow 1.x	PyTorch + Docker + uv

Our contribution is a **deployment stress test** — not just another accuracy leaderboard.

NILMBench2026 at a glance

16

models
classical → Transformer

3

datasets
REDD · UK-DALE · REFIT

2

resolutions
1-min & 15-min

576

configurations
 $16 \times 3 \times 2 \times 6$, ×3 runs

Evaluation philosophy. A deployable NILM model must be **accurate**, **event-aware**, **transferable**, and **efficient** — so we measure all four.

A modern, reproducible software stack

BEFORE

- Legacy TensorFlow / Keras implementations
- Environment drift across papers
- One-off model wrappers
- Accuracy-first reporting

NOW

- Standardized **PyTorch** implementations
- **Docker** and **uv** reproducibility
- A common **NILMTK-compatible** experiment API
- Accuracy, **event detection**, and **compute** metrics

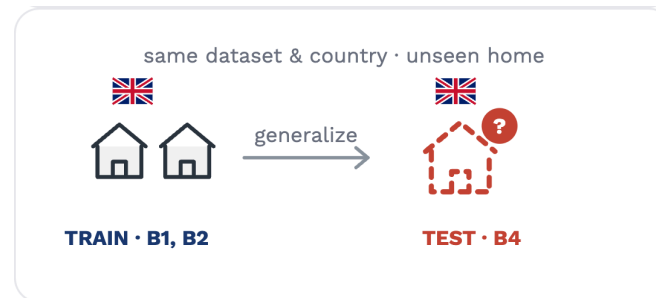
```
uv pip install "nilmtk-contrib[torch] @ git+https://github.com/sustainability-lab/nilmbench.git"
```

Systematic evaluation: three tasks



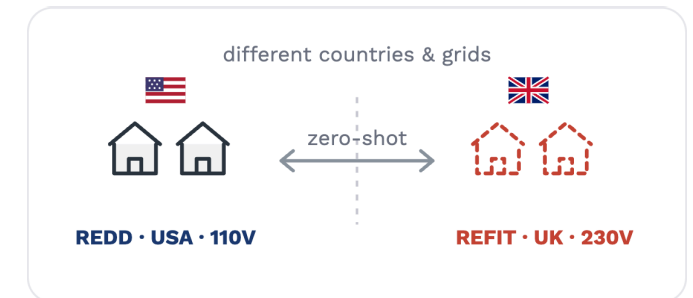
T1 — Same building

Disjoint time windows from one home. A best-case baseline.



T2 — New building

Unseen home, same dataset. Deployment within a region.



T3 — New dataset

Train in one country, test in another. Zero-shot domain shift.

Datasets

DATASET	COUNTRY	BUILDINGS	DURATION	APPLIANCES
REDD	USA — 110 V	6	3–19 days	10–20
UK-DALE	UK — 230 V	5	655 days	5–54
REFIT	UK — 230 V	20	2 years	9–21

Six appliances span the difficulty range: **fridge, microwave, kettle, washing machine, dishwasher, television.**

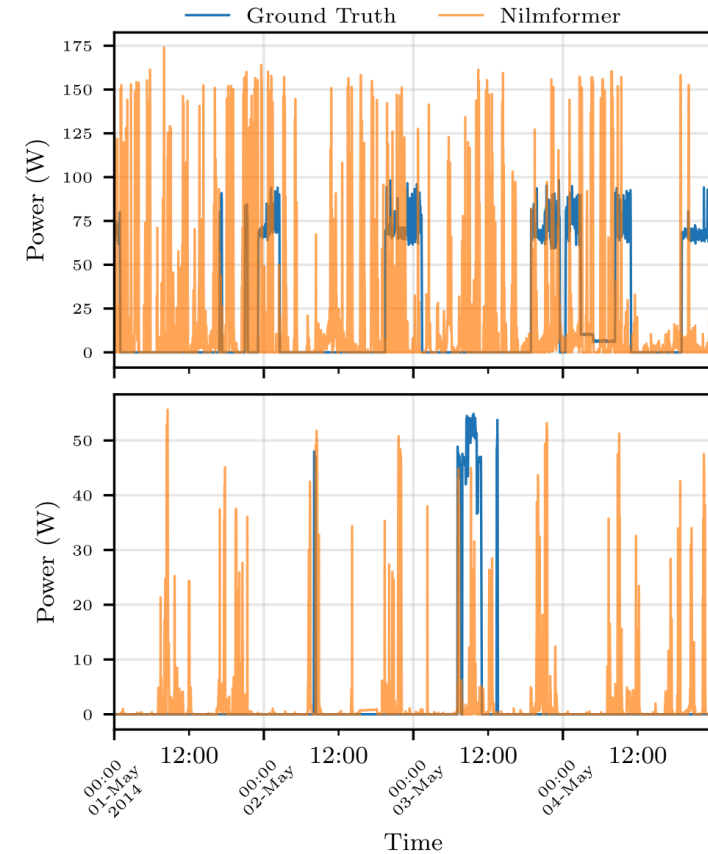
Single-building (AMPds, iAWE, BLUED, DRED) and pay-walled (PecanStreet) datasets are excluded — they cannot support cross-building or cross-dataset evaluation.

Finding 1 — Generalization is the bottleneck

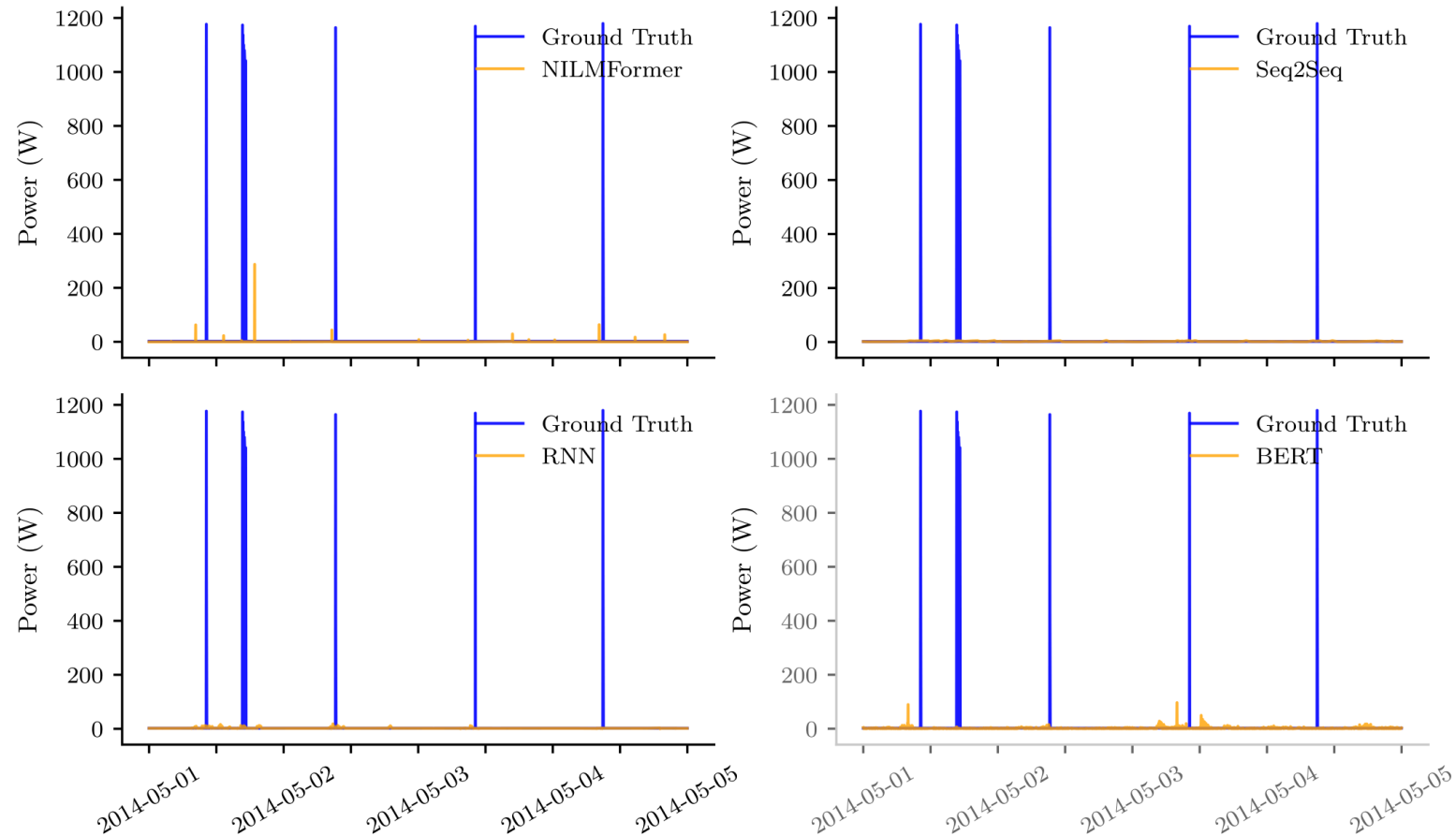
Accuracy degrades sharply from **T1** → **T2** → **T3**, symmetrically in both transfer directions.

- Models learn a **home-specific signature**, not a transferable appliance concept
- Right: NILMFormer tracks a television it was trained on (lower), but **fails on an unseen one** (upper)

The same-building to unseen-building drop is the **core barrier to real-world NILM**.

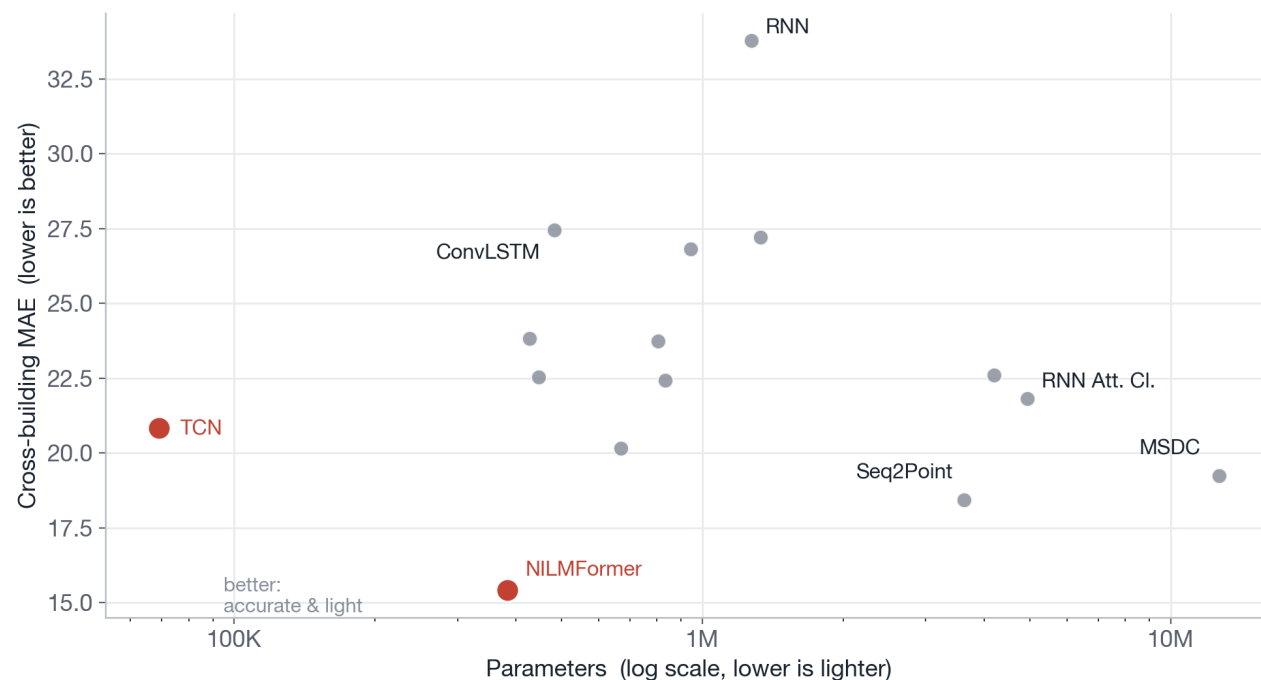


Finding 2 — MAE hides missed appliance events



REFIT microwave (cross-building): all four models miss every high-power activation, yet report a low MAE by predicting near-zero. A near-zero prediction can look acceptable in MAE while missing every event — so we also report **F1**.

Finding 3 — More compute does not guarantee better NILM



The accuracy–compute trade-off is **non-monotonic**.

- **TCN** — 69K params — is competitive with far larger models
- **NILMFormer** — 383K params — is strongest overall
- **RNN Att. Cl.** — 4.94M params — is expensive **and** worse

Architectural inductive bias matters more than raw model size.

How a researcher can contribute

```
from nilmtk.disaggregate import Disaggregator

class MyNILM(Disaggregator):
    def partial_fit(self, mains, appliances): ...
    def disaggregate_chunk(self, mains): ...

experiment['methods']['MyNILM'] = MyNILM({})
```

```
# nilmtk/losses.py – define once
def sae(gt, pred):
    return abs(pred.sum() - gt.sum()) / gt.sum()
experiment['test']['metrics'] += ['sae']
```

A **new algorithm** is one class; a **new metric** is one function.

- The same frozen splits and pre-processing apply to every entry
- Reported gains reflect **architecture**, not implementation differences
- Results are directly comparable on a shared, open leaderboard

NILMBench2026 turns new algorithms and datasets into **comparable results**, quickly.

NILMBench2026: a foundation for deployment

BENCHMARK

16 models · 3 datasets · 2 resolutions · 576 configurations

KEY FINDING

Generalization — not accuracy — is the main bottleneck

PLATFORM

PyTorch + Docker + uv + the NILMTK API

NEXT

Domain adaptation · self-supervised pre-training · edge-ready NILM

Code & project page: github.com/sustainability-lab/nilmbench · sustainability-lab.github.io/nilmbench

Thank you.

Generalization is the bottleneck for real-world NILM. NILMBench2026 is the reproducible platform to measure — and close — that gap.

nipun.batra@iitgn.ac.in · Sustainability Lab, IIT Gandhinagar · sustainability-lab.github.io/nilmbench